

Watching money move

Groundwork does not hold or move funds. **SwyChr does.** What Groundwork needs is the record: for a given client and contractor, that an amount moved on a given date, for a given construction stage — accurate enough to audit at the end of every month. Today three pieces are missing from the API, and none of them can be worked around on our side.

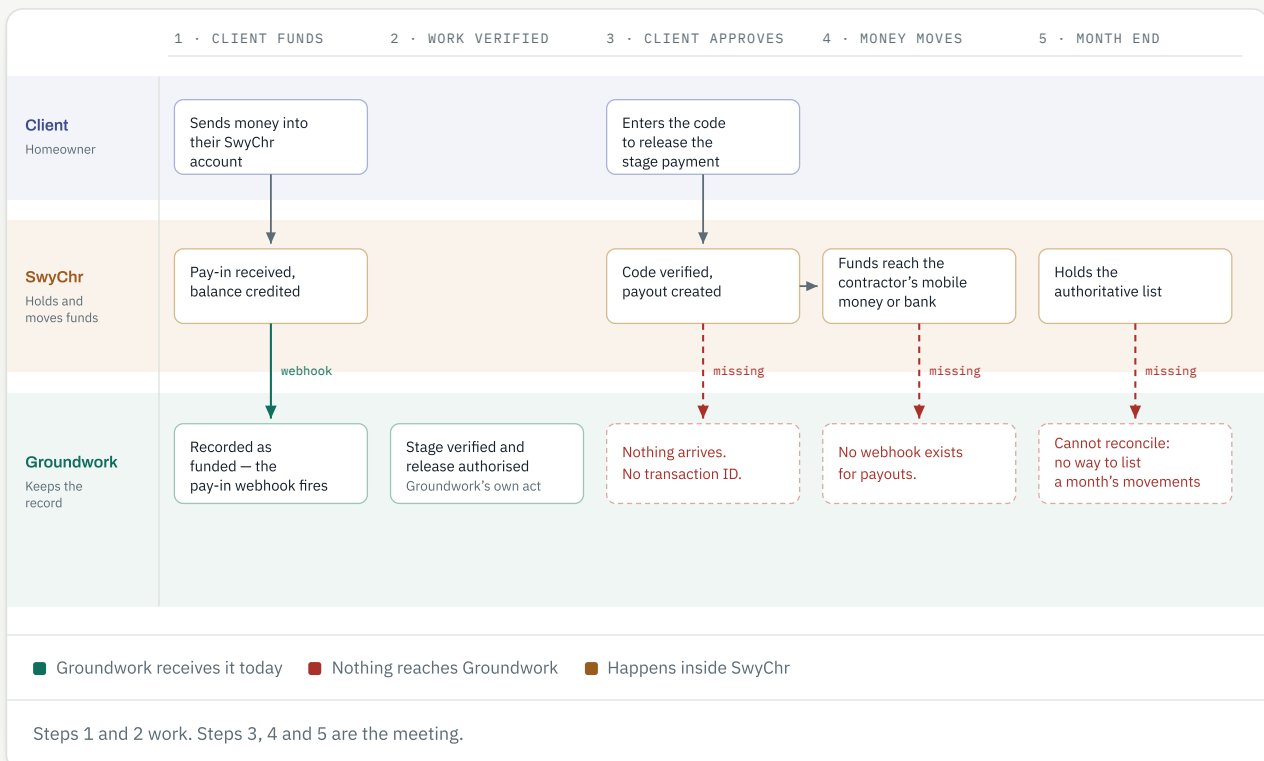
29 September 2026 Prepared for the SwyChr technical meeting Companion to 07-swychr-payout-audit.md

THE ONE THING TO SETTLE IN THE MEETING

If the client authorises and confirms the payout *on SwyChr's platform*, then Groundwork never sees a transaction ID. Every endpoint you publish today answers “what happened to this ID?” — so without a payout webhook and a way to carry our own reference through the transaction, Groundwork cannot attribute a single payout to a project, and a monthly audit is impossible.

01 What actually happens, and where we go blind

The lanes are the three parties. Money only ever moves in SwyChr's lane. Groundwork's lane is the record — and it has holes in exactly the places that matter for an audit.



02 Every endpoint we have today

Eight endpoints. Every one of them either *creates* something or answers a question about a single transaction ID we already hold. None of them lets us discover a transaction we did not create.

ENDPOINT	WHAT IT DOES	FOR MONITORING
/api/payin/admin/auth	Obtain a pay-in JWT	works
/api/payin/create_payment_request	Start a direct pay-in	we may not be the one calling it
/api/payin/create_payment_links	Create a payment link	same
/api/payin/payment_link_status	Status of one payment link	needs an ID we hold
payment_link_status_updated	Pay-in webhook — the only event dispatched	this is the model to copy
/api/payout/admin/auth	Obtain a payout JWT	works
/api/payout/payout_methods	Methods available per country	reference data
/api/payout/create_transaction	Create a payout	not used if SwyChr initiates
/api/payout/transaction_status	Status of one payout	useless without an ID
— no payout webhook —	Nothing is dispatched when a payout settles	blocking
— no list endpoint —	No way to fetch movements by date range	blocking the audit

03 What we are asking for

In priority order. The first three are what make the flow above work at all; the last two make it trustworthy.

1 A payout webhook

blocking

Dispatch an event when a payout changes state — created, sent, settled, failed, reversed — exactly as `payment_link_status_updated` already does for pay-ins. Same signature scheme (HMAC over `{timestamp}. {raw_body}`), same retry policy.

Without this, Groundwork only learns a contractor was paid if a human notices and types the reference in by hand. That does not scale, and it is not auditable.

```
event: payout_status_updated
fields: event_id      unique per delivery, not the transaction id
       transaction_id SwyChr's own id
       external_ref   the reference we supplied (see 3)
       status         numeric, per a published table
       amount, currency
       counterparty    who received it
       occurred_at     when the money actually moved
```

2 A transaction list endpoint

blocking the audit

Fetch every movement on our account between two dates, paginated, filterable by direction and status — both pay-ins and payouts.

Webhooks get missed: an outage on our side, a retry budget exhausted on yours, a deploy at the wrong moment. A webhook tells us something happened; only a list lets us prove our record matches yours. This is the single endpoint that turns “we think we have everything” into a real month-end audit.

```
GET /api/transactions?from=2026-09-01&to=2026-09-30
    &direction=in|out&status=&cursor=
```

```
returns: the same fields as the webhook, plus a stable
        cursor, so a month can be walked exactly once
```

3 Our reference carried through

blocking

Accept a Groundwork-supplied `external_ref` on every transaction — pay-in and payout, whoever initiates it — and return it unchanged on the webhook, on the status response, and in the list.

This is how a movement becomes attributable. Without it we have a list of amounts between phone numbers and no way to say which client, which contractor, or which construction stage any of it belongs to. If the client authorises inside SwyChr's interface, we need a way to attach that reference before the money moves — through the account, the payout request, or a field on the confirmation screen.

4 Counterparty identity we can map

important

On every movement, tell us who was on the other side in a form we can match to our own records: your stable account identifier, plus the masked destination (last four digits of the mobile number or account).

Our audit statement has to read “from this client to this contractor”. A name string alone will not do it — names collide, and ours are often incomplete. A stable identifier lets us map once and trust it afterwards.

5 Published status table and fee treatment

important

The numeric status codes for payouts, with which are terminal — the equivalent of the 0–6 table we already use for pay-ins. And whether your fee is netted from the amount or charged separately.

We map each status onto a ledger state and refuse anything unrecognised rather than guessing. On fees: if the fee is netted, the contractor receives less than the milestone says, and our ledger currently has no way to express that difference.

04 What a month-end audit has to produce

This is the test of whether the integration is finished. At the end of September, Groundwork should be able to generate this without anyone opening SwyChr's dashboard:

- Every movement in the period, in order, with the date and time the money actually moved — not when we recorded it.
- For each one: which client, which contractor, which project and which stage.
- The amount sent, the fee, and the amount received, where those differ.
- A reconciliation line: *our record shows N movements totalling X; SwyChr's list shows the same N totalling the same X* — or a named list of the ones that disagree.

The last line is why ask 2 matters more than it looks. Without a list to compare against, every other number in the report is just our own bookkeeping repeated back to us.

05 Questions to put in the meeting

Q1 Who initiates the payout — SwyChr's interface, or Groundwork calling `create_transaction`?

Everything else follows from this. If it is yours, asks 1 and 3 are non-negotiable. If it is ours, we already hold the ID and only need the webhook.

Q2 Where does the client's confirmation code live, and what does it authorise?

Is it per payout, per session, or per account? Does a successful code produce a transaction we can be told about?

Q3 Is a reversal or refund a new transaction, a state change on the original, or a separate reference?

Our ledger has failed and reconciling states; it has no refunded state yet, and we would rather model yours than invent one.

Q4 If a request times out and we do not know whether you received it, what is the safe recovery procedure?

This is where idempotency stops being theoretical.

Q5 Is there a sandbox?

We will not test a money path against production, and we cannot ship one we have never exercised.

Q6 For Cameroon specifically: which payout methods, which limits, and which bank-code list?

The bank lookup we have seen is Nigeria-only. Cameroon is our primary market.

Groundwork records and governs; SwyChr holds and moves. Nothing in this brief asks SwyChr to change how money moves — only to tell us, reliably and in a form we can reconcile, that it did. Detailed field-by-field mapping against our ledger is in `docs/groundwork-admin/07-swychr-payout-audit.md`.